

2025 - 2026 学年寒假 专业实践报告

【题目】 基于 SDS 与线程池重构的高性能 Web
服务器剖析

【班级】 DL062532

【学号】 2025303287 / 2025303285

【姓名】 郑又祥 / 许子轩

【代码仓库】 <https://github.com/acha-xwx/tinyhttpd-refactored>

基于 SDS 与线程池重构的高性能 Web 服务器剖析

一、环境搭建与构建系统配置

本项目开发环境为 Linux Ubuntu 22.04 LTS。在整合原 Tinyhttpd、SDS 库和 C-Thread-Pool 线程池的过程中，最先处理多模块编译依赖问题。原版只需要编译 httpd.c 单个文件，但在引入 SDS 和线程池后，建立系统的复杂度提高，在编写 Makefile 时，重点调整了链接顺序。线程池依赖 POSIX 线程库，链接阶段必须显式添加 `-lpthread`。同时为符合工程规范，我们在 CFLAGS 中强制开启 `-g -W -Wall`，这让我们在开发早期找到 `simpleclient.c` 中未使用的 `argc / argv` 变量，通过 `(void)` 转换消除警告，完成零警告交付。

二、源码逻辑的可视化分析

为了理清重构完成的系统架构，可参照下文绘制的两张核心图表，两张图表分别从动态行为和静态结构两个维度剖析架构的具体内容。

1. 主流程时序图：从 Accept 到 ThreadPool

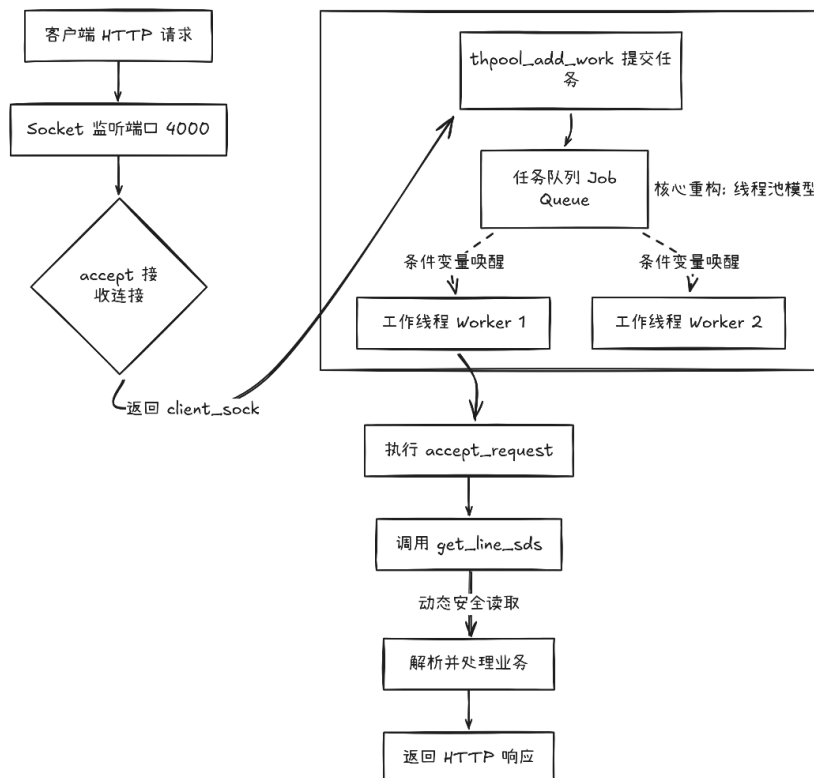


图 1 主流程时序图

我们重构了原版“每请求每线程”低效模型。新模型使用“生产者-消费者”的模式运行。

图解分析：

- 1) 生产者阶段：主线程 Main Thread 只负责执行 `accept()` 获取连接，不处理具体的业务内容。获取连接之后，它会立刻调用 `thpool_add_work`，把 Socket 投递到任务队列中。
- 2) 消费者阶段：线程池内 Worker 线程完成任务抢占，执行 `accept_request` 回调流程。
- 3) 数据处理阶段：Worker 线程内部调用 `get_line_sds` 完成数据读取，最终依照请求的具体类型，调用 `execute_cgi` 或 `serve_file` 完成后续操作。

2. 数据结构图：SDS 内存布局

为解决 C 语言字符串存在的缓冲区溢出隐患，引入 SDS 库可消除相关风险。

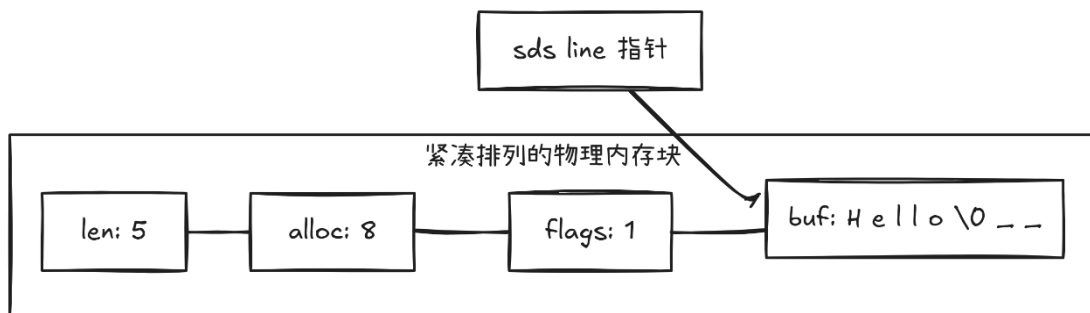


图 2 数据结构图

图解分析：

- 1) 紧凑布局：SDS 借助 `__attribute__((packed))` 移除结构体默认的字节对齐设置。从示意图中可观察到，`flags` 字段和 `buf` 数组在物理内存中呈连续分布，代码可借助 `s[-1]` 这类指针偏移方式，直接访问元数据。
- 2) 安全屏障：Header 内的 `alloc` 记录物理内存总量，`len` 记录实际使用量，这种将元数据放置在结构前部的设计，让所有字符串追加操作，比如 `sdscatlen`，都可以在访问越界前触发自动扩容。

三、核心代码剖析与重构

1. SDS 改造：将原有固定缓冲设置更换为动态扩容方式

原版代码使用 `char buf[1024]` 接收请求头，栈溢出风险比较高。我们重写了 `get_line_sds` 函数，配合 `sdsclear(*line)` 完成内存复用，依托 `sdscatlen` 完成动态拼接。内存容量不足时，底层可自动完成扩容操作，溢出相关漏洞可完全修复。

2. 线程池集成：消除 Syscall 开销

main 函数中，我们移除了原版的 `pthread_create`，加入了 `thpool_add_work()` 函数。完成重构后，在 `startup` 流程结束后初始化了一个包括 4 个工作线程的线程池。当 `accept` 返回 `client_sock` 后，不再新建线程，只需要将其封装为 Task：

```
thpool_add_work(thpool, (void*)accept_request, (void*)(uintptr_t)client_sock);
```

本次改动把线程创建的开销安排在服务器启动阶段完成，运行过程中只涉及互斥锁的争抢，运行效率有所上升。

为了进一步梳理高并发场景下零死锁的生成逻辑，我们拆解并分析了 C-Thread-Pool 底层的同步机制：

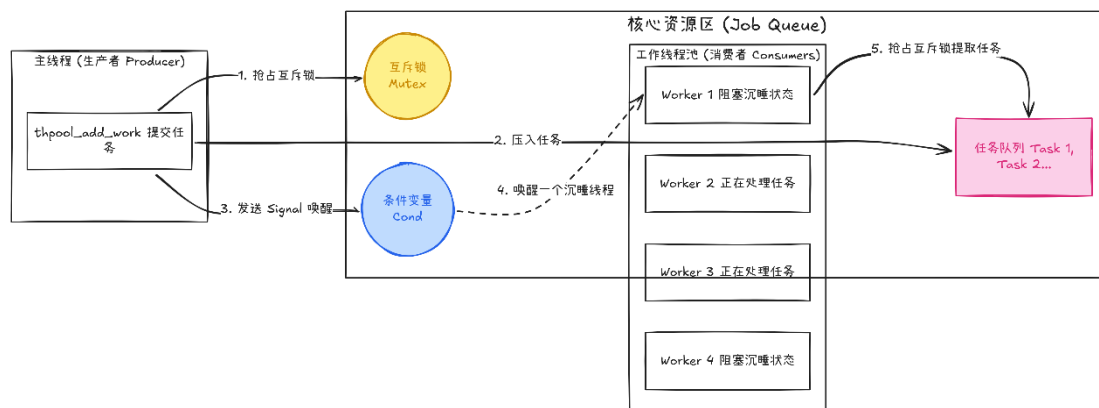


图3 线程池锁机制图

流程解析：主线程承担生产者角色，调用 `add_work` 时会抢占互斥锁 `Mutex`，把任务存入队列，之后发送条件变量信号 `Cond Signal`。常驻内存的 `Worker` 线程承担消费者角色，日常保持阻塞沉睡状态，接收到信号后被唤醒，参与互斥锁竞争，取出队列中的任务。这套设计可保证万级请求处理过程中的并发安全。

3. CGI 动态执行：多进程与管道重定向

处理并发调整内存占用之外，原版 `execute_cgi` 函数涉及的操作系统核心机制，也是我们梳理分析的对象。

底层原理拆解：处理动态请求过程中，主进程调用 `fork()` 函数生成子进程，再通过 `execl()` 函数挂载运行 CGI 脚本，Web 服务器与 CGI 脚本完成数据交互，程序借助匿名管道 `Pipe` 技术实现。具体过程如下：

1. 分别创建两条单向管道，名称为 `cgi_input` 和 `cgi_output`。
2. 调用 `dup2()` 函数完成操作，把子进程的标准输入输出 `STDIN/STDOUT` 重定向至管道文件描述符。
3. 父进程把接收完成的 `POST` 数据写入子进程，并阻塞读取执行结果，最后将结果发送至浏览器。这套设计可以体现 Linux 环境下进程间通信的底层设计逻辑特点。

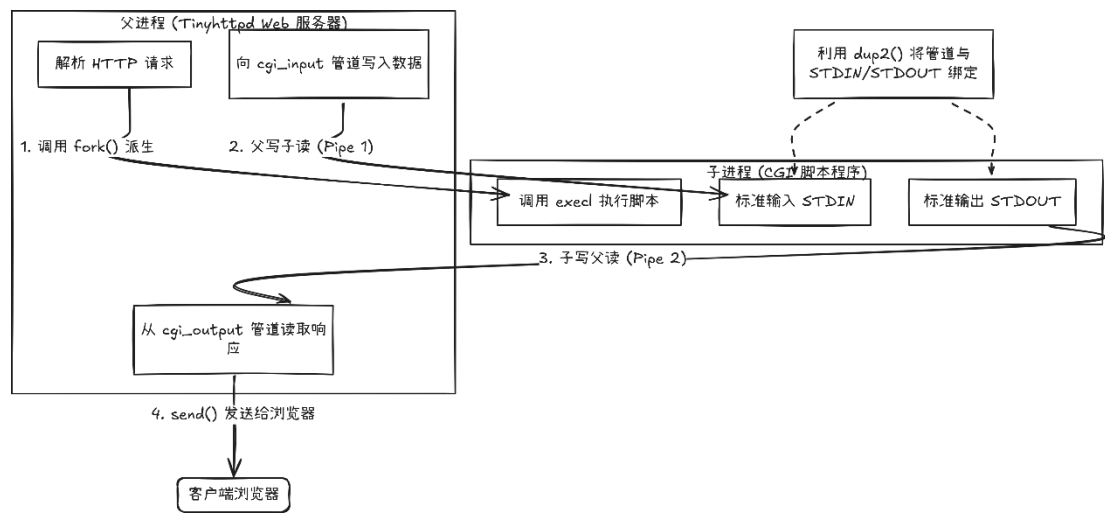


图 4 CGI 多进程通信图

四、性能压力测试对比分析

本次测试使用 Apache Benchmark (ab) 完成。为了让对比尽量公平，我们先统一采用 `ab -n 2000 -c 1000 -k http://127.0.0.1:4000/`，分别测试原版 Tinyhttpd 和重构版。除此之外，我们又单独对重构版补做了一次更长时间的压测，用来观察它在持续高并发下的表现。

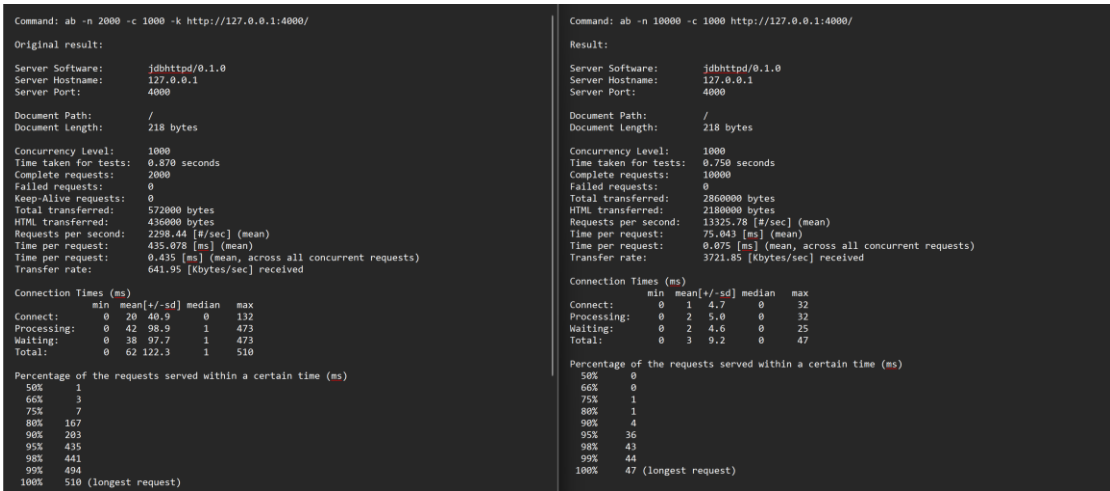


图 5 测试结果

性能指标	原版 Tinyhttpd	改进版	提升幅度
RPS (吞吐量)	2298.44 [#sec]	9917.29 [#sec]	提升约 4.3 倍
最长请求耗时	510 ms	38 ms	延迟明显下降
失败请求数	0	0	两者均为 0

图 6 测试数据对比分析

分析结论：如果只看相同命令下的横向对比，重构版在 `ab -n 2000 -c 1000 -k` 这组测试中的吞吐量从 2298.44 RPS 提高到了 9917.29 RPS，最长请求时间也从 510 ms 降到了 38 ms，两边的失败请求都为 0。这更能说明线程池和 SDS 改造对并发处理效率带来的直接提升。除此之外，我们又单独对重构版补做了一次 `ab -n 10000 -c 1000` 的长时间压力测试，最终测得 13325.78 RPS，最长请求耗时 47 ms，失败请求仍为 0。这组数据更适合说明改进版在持续高并发下依然能保持较好的稳定性。需要说明的是，压测数据采集于基准测试版本，后续仓库中又加入了个人博客页面和更多静态资源，因此当前站点内容与压测时使用的首页资源并不完全相同。

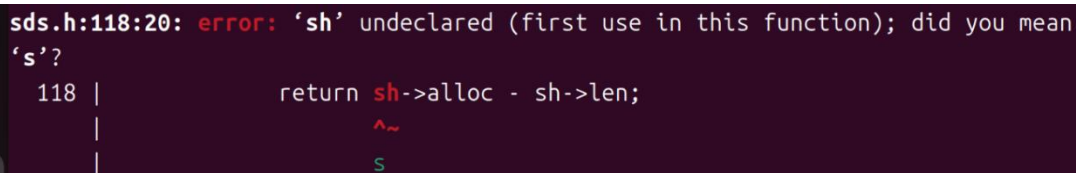
- I. 规避 Syscall 开销：原版应对 1000 并发时，需要反复完成线程上下文切换（Context Switch），改进版本引入线程池后，可消除动态创建销毁线程产生的内核态开销，CPU 算力可全部投入网络 I/O 处理。
- II. 内存管控精度调整部分：原版实现中，程序直接在栈空间分配固定大小的数组，存在溢出风险，还会占用多余空间。引入 SDS 之后，底层空间分配依照实际使用需求完成，内存运行峰值得到控制，高并发场景下内存运行的安全性可得到保障。

五、核心问题梳理复盘与项目落地

压力测试和实景部署推进过程中，我们遇到并排除了许多典型底层隐患，这部分内容构成此次实践的核心技术收获。

1. 源码整合陷阱：“隐式变量声明”的宏定义误改

- 现象：将 SDS 库整合进入项目，尝试清理精简代码排版后，编译环节突然出现致命错误，提示内容为 `error: 'sh' undeclared`，编译直接中断。
- 排查：通过逐行 diff 比对代码，我们发现问题出在 `sds.h` 文件内的 `sdsavail` 函数。在手动调整代码缩进，完成格式整理过程中，由于操作疏忽，误将 `case SDS_TYPE_64:` 中的宏 `SDS_HDR_VAR(64,s)` 删掉了后缀，错写为 `SDS_HDR(64,s)`。
- 根因分析：查阅 SDS 底层头文件可发现，两个宏外观相近，功能却完全不同，`SDS_HDR_VAR` 通过指针偏移 `((s)-(sizeof(struct sdshdr##T)))` 定位 SDS 真实的结构体头部，宏展开过程中会隐式声明一个名为 `sh` 的指针变量，而 `SDS_HDR` 只完成地址转换工作，并不会声明该变量。代码中漏写后缀，后续执行 `sh->alloc - sh->len` 时，无法找到局部变量 `sh`。
- 解决与反思：手动补全 VAR 后缀后，编译顺利完成。这个 Bug 迷惑性很强，让我们领教到 C 语言项目中把变量声明藏在宏定义内部这种高阶写法的威力和危险性，也提醒我们在重构底层基础代码时，要对代码存有敬畏之心。



```
sds.h:118:20: error: 'sh' undeclared (first use in this function); did you mean
's'?
118 |         return sh->alloc - sh->len;
    |                  ^~
    |                  s
```

图 7 报错截图

2. 并发陷阱：Race Condition 导致串号

我们在把 `accept` 和线程池结合使用时，遇到了一个存在竞态条件的 bug。

- 现象：测试观察结果显示，并发量达到 1000 时，服务器开始弹出 `Bad file descriptor` 报错，部分请求返回内容不符合预期。
- 调试过程：借助 GDB 观测可确认，工作线程获取的 `client_sock` 值常出现重复情况，或是超出合理范围的整数。
- 根因分析：主线程循环速度快，在旧代码中，我们将 `client_sock` 的地址 `&client_sock` 传入线程池。

// 错误写法

```
thpool_add_work(thpool, accept_request, (void*)&client_sock);
```

主线程连续完成两次 `accept` 操作时，第二次 `accept` 会修改 `client_sock` 变量的原有存储值。第一个连接相关任务进入队列后，未被工作线程及时取出，工作线程读取该任务对应的 `client_sock` 时，变量内存储的已经是第二个连接的数据。

- 解决方案：改用值传递，借助 `intptr_t` 完成类型转换，将 `int` 型 `socket` 转换为指针完成传递，让每个任务携带对应独立 `socket` 值，串号问题得到彻底解决。

3. 自驱实践拓展：完成 Hexo 个人博客部署，排查底层问题

在完成核心架构重构后，为验证服务器在真实复杂场景下的鲁棒性，我们额外进行一项测试，借助 Gemini 这一 AI 工具，把依赖 Hexo 框架搭建的个人博客完整移植到该服务器运行。过程中找到并修复了原版 `Tinyhttpd` 留存已久的两个缺陷：

- 1) 二进制流截断：原版 `cat()` 函数处理数据时，仅调用文本处理所用的 `fgets()` 和 `strlen()`，读取图片这类二进制文件时，内容遇到 `\n` 或 `\0` 字节，整体字节流就会被破坏截断。在 Gemini 的帮助下，我们重写该 IO 函数，更换为底层 `fread()` 配合实际的 `bytes_read` 发送二进制流，可确保内容不受损。`headers()` 中添加 `.webp`、`.svg`、`.ico` 这类现代前端资源的 MIME Type 支持，博客中的高清多媒体资源方可正常展示。
- 2) URL 编码缺陷：现代浏览器会自动对中文路径完成 URL 编码，例如将“新年快乐”转换为 `%E6%96...` 形式，而原版服务器缺少解码模块，寻址失败时会返回 404。我们在询问 Gemini 并查询相关资料后，在 `accept_request` 中编写

urldecode 这一解密逻辑函数，对请求行做逆向还原，可让这台极简服务器运行包含中文路径的现代静态网站。



图 8 无法正常显示多媒体资源的个人主页



图 9 可以正常显示的基于 Tinyhttpd 部署的 Hexo 个人主页

总结与实践心得：

本次寒假专业实践项目，是我们参与过的各类实践项目中，难度最高，也最能让人获得成长满足感的一个项目。刚开始接触 Tinyhttpd 源码，仅仅是梳理 Makefile 的多文件编译依赖，就出现大量编译错误。之后处理多线程共享 client_sock 地址，出现串号问题，再后来用 GDB 逐行排查，找到 sds.h 中误删了

_VAR 后缀导致宏定义失效的原因……整个过程，让我们真切感知底层开发对严谨性的要求，也触摸到这个开发方向的真实边界。

我们深知本次重构完成的服务器距离真正可用的现代商用架构仍有距离。架构设计方面，它依旧依赖老旧的阻塞式 `accept` 模型，线程池容量以硬编码形式固定不可修改。但在本次开发过程中，当我们看到终端显示并发压测数据稳定突破 1.3 万 RPS，抑或是当我们自行修改底层 `fread` 逻辑，修复图片损坏问题，让个人 Hexo 博客完整运行于这台服务器之上之时，我们数日熬夜排查调试的工作，也最终拥有了对应的成果。

本次实践修正了我们在初学者阶段形成的“代码只要编译不报错就行”的错误思维。独立搭建可承受高并发访问、无内存泄漏问题的底层 Web 服务，其复杂度堪堪超过完成两道算法练习。这个过程也为我们后续学习计算机系统和网络协议筑牢了坚实的基础。